Notes on

Let's Go!

Basics

- Handlers for the application are like MVC controllers: they handle app logic and HTTP requests
- Routers are called servemux in Go
- Servemux is a HTTP request multiplexer
- It matches the URL of each incoming request against a list of registered patterns and calls the handler for the pattern that most closely matches the URL
    - It stores a map of URL patterns to handlers
- The default servemux is used to set the default handlers for the server
- But you also need a web server to listen and serve the requests
- Use the http.NewServeMux() function to initialize a new servemux, then register the home function as the handler for the ”/” URL pattern (for example).
    - Always use the http.NewServeMux() function to create a new servemux, rather than using the default servemux provided by the http package.
- Use the http.NotFound() function to send a 404 response
- The http.Request value is a pointer to the current HTTP request, and it’s a struct with various fields and methods for working with the request
- http.Error(w, "Method Not Allowed", http.StatusMethodNotAllowed) is the same as w.WriteHeader(405) and w.Write([]byte("Method Not Allowed"))

Fixed and subtree paths

Go’s servemux supports two types of URL patterns: fixed and subtree paths.
Fixed paths don’t end with a slash, and subtree paths do end with a slash.

Fixed path patterns match the exact URL pattern, whereas / is a subtree path pattern - it matches any URL that has / as a prefix, such as /, /foo, /foo/bar, etc.
So it's like a catch-all pattern.

Serving

Use the http.ListenAndServe() function to start a new web server.

It requires two parameters: the TCP network address to listen on (in this case ":4000") and the servemux to use (in this case mux).
Using:``4000 is fine - the server will listen on all available network interfaces.

If you want to listen on a specific network interface, you can use the IP address of the interface as the first parameter to http.ListenAndServe().

In some projects, you may see :http or :http-alt instead of :4000 - these are the names of the services in the /etc/services file

Fatal Logging

Go's stdlib log.Fatal() writes the error message and then exits the program (same as log.Println() followed by a call to os.Exit(1)).

http.Handler

What is a handler?

First, it's an object that satisfies the http.Handler interface.
So it needs to have a serveHTTP(http.ResponseWriter, *http.Request) method.
That could be the simple version like this:

```
type home struct {}


func (h *home) ServeHTTP(w http.ResponseWriter, r *http.Request) {
	w.Write([]byte("Home page"))
}


// That's a handler. So we can register it with a servemux:
mux := http.NewServeMux()
```

mux.handle("/", &home{})

But in practice, we don't create objects, just to make a ServeHTTP method on them. We just write the handler as a function. But now it doesn't fulfill the http.Handler interface — there's no object with a ServeHTTP method! That's what we use http.HandlerFunc() for. We transform the lone function into a handler, using the http.HandlerFunc() adapter:

func home(w. http.ReponseWriter, r *http.Request) { ... }

mux := http.NewServeMux()

mux.Handle("/", http.HandlerFunc(home))

Handlers can be chained.

http.ListenAndServe() takes an address as its first parameter, and a Handler as its second parameter. But we pass in a servemux. We can do that because servemux also has a ServeHTTP method, meaning it satisfies http.Handler.

A servemux is like a special kind of handler that, instead of responding to a request, passes it to a second handler. This is chaining handlers, a common concept in Go.

When our server receives a new HTTP request, it calls the servemux's ServeHTTP, which looks up the relevant handler based on the URL path. It then calls that handler's ServeHTTP.

A Go web application can be thought of as a chain of ServeHTTTP methods being called one after another.

Requests are handled concurrently.

They are handled in their goroutine, meaning they are handled concurrently.
This makes Go fast but also necessitates that you look out for race conditions when accessing shared resources from your handlers.

Managing Configuration Settings

Example: Adding a configurable address CLI flag

```
package main

import (

        "flag" // import this
```

```
	"log"
	"net/http"
)
func main() {
	// Define flag, default value, and description
	addr := flag.String("addr", ":4000", "HTTP network address")
	// Parse flags --- will have default value otherwise
	flag.Parse()

	// ...

	// The flag variable is a pointer, so dereference it
	log.Printf("Starting server on %s\n", *addr)
	if err := http.ListenAndServe(*addr, mux); err != nil {
		log.Fatal(err)
	}
}
```

Here we use flag.String(). It'll convert the value of whatever's passed to a string type, but will also log an error and exit if that isn't possible. There's also flag.Int(), flag.Bool(), flag.Float64(), which does the same, but for their types. You'll automatically get a -help` flag.

Environment variables

```
// Snippetbox is the name of the project
addr := os.Getenv("SNIPPETBOX_ADDR")
```

When using env vars, you can't specify default values, you don't get -help, and the value is always a string.
It's probably better to pass environment variables as strings when starting the app:

```
export SNIPPETBOX_ADDR=":9999"
```

go run cmd/web/* -addr=$SNIPPETBOX_ADDR

Passing configuration to pre-existing variables

You can parse CLI flags to memory addresses of existing variables using flag.StringVar() (and the other ones with Var at the end).
This is good for storing your configuration settings in a single struct, for example.

```
type Config struct {
	Addr string
	StaticDir string
}


cfg := new(Config)
flag.StringVar(&cfg.Addr, "addr", ":4000", "HTTP network address")
flag.StringVar(&cfg.StaticDir, "static-dir", ":4000", "Path to static assets")
flag.Parse()
```

Structured Logging

Logs can have levels, like Info, Debug, Warn, and Error. This defines the severity of the message.

Go's stdlib has log/slog which lets you create custom structured loggers.
Each log entry would have:

- **Timestamp with ms precision**
- **Severity**
- **Message**
- **Optionally, attributes (key-value pairs) with additional information**

You can define new loggers:

```
import (
	"log/slog"
	"os"
```

```
	"net/http"
)

func main() {
	// ...
	// There's also a JSON handler
	logger := slog.New(slog.NewTextHandler(os.Stdout, nil))

	// ...
	logger.Info("starting server", slog.Any("addr", *addr))
	if err := srv.ListenAndServe(); err != nil {
		logger.Error(err.Error())
		os.Exit(1)
	}
}
```

In a staging or production environment, you may want to redirect the streams to another destination (e.g. on-disk, or a logging service). Here's how to do it on-disk:

```
go run cmd/web/* >>/tmp/info.log 2>>/tmp/error.log
```

Logging rules of thumb

Custom loggers by slog.New() are concurrency-safe. You can share it and use it around multiple goroutines.

But beware if they write to the same destination – the destination's underlying Write() should be safe for concurrent use as well.

Dependency Injection

There are many ways to do this, but one way is to inject dependencies into your handlers.

For applications where all handlers are in the same package, you can create a custom application struct and define handler functions as methods against application.

```go
// cmd/web/main.go
type application struct {
	logger *slog.Logger
}

func main() {
	// ...
	app := &application{
		logger: logger,
	}

	// ...

	logger.Info("starting server", slog.Any("addr", *addr))
	if err := http.ListenAndServe(*addr, app.routes()); err != nil {
		logger.Error(err.Error())
		os.Exit(1)
	}
}

// cmd/web/routes.go
func (app *application) routes() *http.ServeMux {
	mux := http.NewServeMux()

	// ...
	mux.HandleFunc("/", app.home)
	// ...
```

```
	return mux
}



// cmd/web/handlers.go
func (app *application) home(w http.ResponseWriter, r *http.Request) {
	// ...
	if err != nil {
		app.errorLog.Println(err.Error())
		http.Error(w, "Internal Server Error",
http.StatusInternalServerError)
	return
}
```

But what if handlers are spread across multiple packages?
Then you could create a config package exporting an Application struct, and have your handler functions close over this to form a *closure*.

Using databases

This book uses mysql with the github.com/go-sq–driver/mysql driver.

db, err := sql.Open("driver-name-here", "connection-string-here (data source name)")

- **You should put parseTime=true as a query parameter on your connection string.**
- **db is a sql.SB object, not a database connection. It's a pool of many connections.**
- **The connections in the pool are managed as needed, meaning it automatically opens and closes database connections via the driver.**
- **The connection pool is safe for concurrent access.**

- **It’s normal to initialize the connection pool in main and pass it to your handlers. It’s supposed to be long-lived. Don’t waste memory and network resources on sql.Open() in individual handlers.**

Connecting to the database

```
// cmd/web/main.go

import (

	// Imported with an underscore to avoid the "imported and not used" error

	// We need to import the driver package to register the driver with the database/sql package

	// The driver package will be used implicitly by the sql.Open() function

	_ "github.com/go-sql-driver/mysql"

)


func openDB(dsn string) (*sql.DB, error) {

	db, err := sql.Open("mysql", dsn)

	if err != nil {

		return nil, err

	}


	// `sql.Open` doesn't start connections, so we check if we can access it here

	if err := db.Ping(); err != nil {

		return nil, err

	}


	return db, nil

}
```

```
func main() {
	// ...
	db, err := openDB(*dsn)
	if err != nil {
		logger.Error(err.Error())
		os.Exit(1)
	}

	defer db.Close()
	// ...
}
```

Model / Service Layer / Data Access Layer

Put your models in internal/models/.

For example:

```
// internal/models/snippets.go
package models

import (
	"database/sql"
	"time"
)

type Snippet struct {
	ID      int
	Title   string
	Content string
```

```
	Created time.Time
	Expires time.Time
}

type SnippetModel struct {
	DB *sql.DB
}

// Insert will insert a new snippet into the database.
func (m *SnippetModel) Insert(title, content, expires string) (int, error) {
	return 0, nil
}

// Get will return a specific snippet based on its id.
func (m *SnippetModel) Get(id int) (Snippet, error) {
	return Snippet{}, nil
}

// Latest returns the 10 most recently created snippets.
func (m *SnippetModel) Latest() ([]*Snippet, error) {
	return nil, nil
}
```

Now you can extend your application struct and pass around the model with dependency injection:

```
// cmd/web/main.go
package main

import (
```

```
	"database/sql"
	"flag"
	"log/slog"
	"net/http"
	"os"

	"github.com/chhoumann/snippetbox/internal/models"

	// Imported with an underscore to avoid the "imported and not used" error
	// We need to import the driver package to register the driver with the database/sql package
	// The driver package will be used implicitly by the sql.Open() function
	_ "github.com/go-sql-driver/mysql"
)

type application struct {
	logger   *slog.Logger
	snippets *models.SnippetModel
}

func main() {
	addr := flag.String("addr", ":4000", "HTTP network address")
	dsn := flag.String("dsn", "root:my-secret-pw@/snippetbox?parseTime=true", "MySQL data source name")
	flag.Parse()
```

```
	logger := slog.New(slog.NewTextHandler(os.Stdout,
&slog.HandlerOptions{}))

	db, err := openDB(*dsn)
	if err != nil {
		logger.Error(err.Error())
		os.Exit(1)
	}

	defer db.Close()

	app := &application{
		logger:   logger,
		snippets: &models.SnippetModel{DB: db},
	}

	logger.Info("starting server", slog.Any("addr", *addr))
	if err := http.ListenAndServe(*addr, app.routes()); err != nil {
		logger.Error(err.Error())
		os.Exit(1)
	}
}

func openDB(dsn string) (*sql.DB, error) {
	// ...
}
```

Using SQL

You can do

- **DB.Query() for selects returning multiple rows**
- **DB.QueryRow() for selects only returning a single row**
- **DB.Exec() for statements that don't return rows (e.g. INSERT, DELETE)**

Example INSERT

```
// Insert will insert a new snippet into the database.
func (m *SnippetModel) Insert(title, content, expires string) (int, error) {
	// Using the ? placeholder for the values we want to insert
	// This is a feature of the Go MySQL driver
	// It will automatically escape the values to prevent SQL injection attacks
	// The placeholders will be replaced by the actual values in the same order as they appear in.
	stmt := `INSERT INTO snippets (title, content, created, expires)
	VALUES(?, ?, UTC_TIMESTAMP(), DATE_ADD(UTC_TIMESTAMP(), INTERVAL ? DAY))`

	// Exec() method to execute the statement
	// It returns a sql.Result object which contains some basic information about what happened
	result, err := m.DB.Exec(stmt, title, content, expires)
	if err != nil {
		return 0, err
	}

	// LastInsertId() method to get the ID of the newly inserted record
	id, err := result.LastInsertId()
	if err != nil {
		return 0, err
```

```
    }

    // Convert the ID from an int64 to an int before returning
    return int(id), nil
}
```

Here's how the placeholder parameters work to prevent SQL injection attacks from untrusted user-provided input with DB.Exec():

1. **Create a prepared statement on the db with the SQL statement you provide. This is parsed and compiled by the db, and then stored for execution.**
2. **Pass the parameter values to the db. The db executes the prepared statement with these parameters. Because they were transmitted after the statement has been compiled, they're treated as pure data by the db. So they don't change the intent of the statement, thereby preventing potential attacks.**
3. **Close / deallocate the prepared statement on the db.**

Example SELECT (single)

```
// internal/models/snippets.go
// Get will return a specific snippet based on its id.
func (m *SnippetModel) Get(id int) (Snippet, error) {
    stmt := `SELECT id, title, content, created, expires FROM snippets
    WHERE expires > UTC_TIMESTAMP() AND id = ?`

    row := m.DB.QueryRow(stmt, id)

    var s Snippet // Initialize a new Snippet struct to hold the data

    // Copy values from each field into the Snippet struct
    if err := row.Scan(&s.ID, &s.Title, &s.Content, &s.Created,
&s.Expires); err != nil {
```

```
		// If the query returns no rows, then row.Scan() will return a
sql.ErrNoRows error
		if errors.Is(err, sql.ErrNoRows) {
			return Snippet{}, ErrNoRecord // this is a custom error
made with `errors.New`.
		} else {
			return Snippet{}, err
		}
	}

	return s, nil
}

// cmd/web/handlers.go
func (app *application) snippetView(w http.ResponseWriter, r
*http.Request) {
	id, err := strconv.Atoi(r.URL.Query().Get("id"))
	if err != nil || id < 1 {
		app.notFound(w)
		return
	}

	snippet, err := app.snippets.Get(id)
	if err != nil {
		if errors.Is(err, models.ErrNoRecord) {
			app.notFound(w)
		} else {
			app.serverError(w, r, err)
```

```
		}
		return
	}

	fmt.Fprintf(w, "%+v", snippet)
}
```

Example SELECT (many)

```
// Latest returns the 10 most recently created snippets.
func (m *SnippetModel) Latest() ([]Snippet, error) {
	stmt := `SELECT id, title, content, created, expires FROM snippets
	WHERE expires > UTC_TIMESTAMP() ORDER BY created DESC LIMIT 10`

	rows, err := m.DB.Query(stmt)
	if err != nil {
		return nil, err
	}
	// Ensure resultset is closed before `Latest` returns. This needs to come after the error check.
	// Keeping the resultset open would keep the connection open, and could lead to resource leaks
	defer rows.Close()

	var snippets []Snippet

	// Iterate through the resultset
	// Use rows.Next() to prepare the first (and subsequent) row(s) for scanning
```

```
	// If there are no rows, or an error occurs, rows.Next() will return false, terminating the loop
	// The resultset automatically closes when we've iterated over all the rows (+ frees up db connection)
	for rows.Next() {
		var s Snippet
		if err := rows.Scan(&s.ID, &s.Title, &s.Content, &s.Created, &s.Expires); err != nil {
			return nil, err
		}
		snippets = append(snippets, s)
	}

	// Check for errors during iteration - can't assume successful iteration
	if err = rows.Err(); err != nil {
		return nil, err
	}

	return snippets, nil
}
```

Tips for database usage

- **If you're tired of sql driver verbosity:**
  - [GitHub - blockloop/scan: Tiny lib to scan SQL rows directly to structs, slices, and primitive types]
  - [GitHub - jmoiron/sqlx: general purpose extensions to golang's database/sql]
- **Go doesn't handle NULL values in database records well. Try to avoid NULL values altogether – set sensible defaults.**
- **Transactions**

    - **To guarantee the same connection is used for your SQL, you can use transactions.**
    - **Start with tx, err := DB.Begin() to "start a transaction" — remember to check for errors.**
    - **Do defer tx.Rollback(). If there's an error, this rolls back the changes before the function you're in returns. Otherwise, if the transaction succeeds, it'll be committed by the time this is called, making it a no-op.**
        - **So either all statements execute successfully, or none are executed and the db remains unchanged.**
    - **Execute your statements with tx.Exec. Check for errors.**
    - **Commit with tx.Commit() (can return an error).**
- **Prepared statements**
    - **It may be better to use DB.Prepare to create your own prepared statements and reuse those, instead of the ones Exec, Query, and QueryRow use behind the scenes.**
        - **Especially for complex SQL statements (e.g. multiple JOINs) and those that are repeated often (re-preparing costs).**
        - **You can store the prepared statement in your model struct as *sql.Stmt, create a model constructor where you use db.Prepare, and then use it in your data access layer functions.**
        - **But prepared statements exist on database connections. Because there can be multiple connections in a pool, this would often just result in the statement being recreated on each new connection. You could even run into the server-side limits on the number of statements.**
        - **So preferring Query, QueryRow, and Exec over preparing statements manually is a fine starting point. You can adjust when it's worth preparing statements yourself.**

Middleware

Middleware is just another handler you insert into your handler chain. You can use it for e.g. logging.

Here's a common pattern to create middleware:

```
func myMiddleware(next http.Handler) http.Handler {
	return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
		// Do stuff before the next handler is called
		next.ServeHTTP(w, r)
		// Do stuff after the next handler is called
	})
}
```

It's important to position your middleware right.
If you place it before the servemux, it'll run on every request. That's good for e.g. logging, which you'd want for every request.
If you put it after the servemux, it'll run for a specific route. That would be good for e.g. authorization, which may only be needed on some routes.

Setting security headers

You may want to set some security headers, e.g.:

Content-Security-Policy: default-src 'self'; style-src 'self' fonts.googleapis.com; font-src fonts.gstatic.com

Referrer-Policy: origin-when-cross-origin

X-Content-Type-Options: nosniff

X-Frame-Options: deny

X-XSS-Protection: 0

- **Content-Security-Policy (CSP) restricts where the resources for your page can be loaded from. Helps prevent XSS, clickjacking, and other code-injection attacks.**
- **Referrer-Policy controls what information is included in a Referer header when users navigate away from your site. origin-when-cross-origin means the full URL is included for same-origin requests, but on all other requests, URL path and query string values are removed.**

- **X-Content-Type-Options: nosniff tells browsers not to MIME-type sniff the content type of the response. Helps prevent content-sniffing attacks.**
- **X-Frame-Options: deny helps prevent clickjacking attacks in older browsers that don't support CSP headers.**
- **X-XSS-Protection: 0 is used to disable blocking of XSS attacks. When using CSP, it's better to disable this.**

The middleware for that would look like this:

```
// cmd/web/middleware.go

package main


import (

	"net/http"

)


func secureHeaders(next http.Handler) http.Handler {

	return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {

		w.Header().Set("Content-Security-Policy", "default-src 'self';
style-src 'self' fonts.googleapis.com; font-src fonts.gstatic.com")


		w.Header().Set("Referrer-Policy", "origin-when-cross-origin")

		w.Header().Set("X-Content-Type-Options", "nosniff")

		w.Header().Set("X-Frame-Options", "deny")

		w.Header().Set("X-XSS-Protection", "0")


		next.ServeHTTP(w, r)

	})

}
```

```
// USAGE
func (app *application) routes() http.Handler {
	mux := http.NewServeMux()

	// ...

	return secureHeaders(mux)
}
```

Early returns

You can do early returns in your middleware (before next.ServeHTTP) to stop the chain from being executed and have control flow back upstream.

Authentication is a common example:

```
func myMiddleware(next http.Handler) http.Handler {
	return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
		if (!isAuthorized(r)) {
			w.WriteHeader(http.StatusForbidden)
			return
		}

		next.ServeHTTP(w, r)
	})
}
```

Logging middleware

```
// cmd/web/middleware.go
func (app *application) logRequest(next http.Handler) http.Handler {
```

```
	return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
		var (
			ip     = r.RemoteAddr
			proto  = r.Proto
			method = r.Method
			uri    = r.RequestURI
		)

		app.logger.Info(
			"received request",
			slog.Any("ip", ip),
			slog.Any("proto", proto),
			slog.Any("method", method),
			slog.Any("uri", uri),
		)

		next.ServeHTTP(w, r)
	})
}
```

Panic Recovery

By default, the server will assume panics in handlers shouldn't crash the entire server. So they don't. However, if it does occur, the user doesn't get anything but an empty reply unless you handle it.
This isn't great, so you can handle it with this middleware:

```
// cmd/web/middleware.go

func (app *application) recoverPanic(next http.Handler) http.Handler {
	return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
```

```
		// Because of defer, this runs during the stack unwinding when panics occur
		defer func() {
			if err := recover(); err != nil {
				w.Header().Set("Connection", "close")
				app.serverError(w, r, fmt.Errorf("%s", err))
			}
		}()

		next.ServeHTTP(w, r)
	})
}
```

This only recovers panics that happen in the same goroutine that executed the middleware. So if your handler spun up another goroutine (e.g. for background processing), then any panics in that second goroutine aren't recovered by this middleware, not the built-in panic recovery in the Go HTTP server: they'd cause a server crash.
So make sure you handle this. If your goroutines can cause panics, use something like this:

```
// inside handler function

// Spin up a new goroutine to do some background processing.
go func() {
	defer func() {
		if err := recover(); err != nil {
			app.logger.Error(fmt.Sprint(err))
		}
	}()

	doSomeBackgroundProcessing()
```

}()

// ...

Composable middleware chains

justinas/alice is a great package for managing middleware/handler chains.

// Instead of

return myMiddleware1(myMiddleware2(myMiddleware3(myHandler)))

// You do

return alice.New(myMiddleware1, myMiddleware2, myMiddleware3).Then(myHandler)

// Or assign to variables, append to, reuse:

myChain := alice.New(myMiddlewareOne, myMiddlewareTwo)

myOtherChain := myChain.Append(myMiddleware3)

return myOtherChain.Then(myHandler)

Forms

To receive form data from POST requests, you can do something like this:

// cmd/web/handlers.go

```
func (app *application) snippetCreatePost(w http.ResponseWriter, r *http.Request) {
	err := r.ParseForm() // adds any data in post request bodies to r.PostForm map
	if err != nil {
		app.clientError(w, http.StatusBadRequest)
		return
	}
```

```
	title := r.PostForm.Get("title")
	content := r.PostForm.Get("content")
	expires, err := strconv.Atoi(r.PostForm.Get("expires"))
	if err != nil || expires < 1 || expires > 365 {
		app.clientError(w, http.StatusBadRequest)
		return
	}

	id, err := app.snippets.Insert(title, content, expires)
	if err != nil {
		app.serverError(w, r, err)
		return
	}

	http.Redirect(w, r, fmt.Sprintf("/snippet/view/%d", id),
http.StatusSeeOther)
}
```

The r.PostForm map works, but you could also use r.Form.
r.PostForm is populated only for POST, PATCH, and PUT requests and has the form data from the request body.
r.Form is populated for all requests and has the form data from the request body *and* any query string parameters. If there are conflicts, the request body value takes precedence.

If you have a form field that sends multiple values (e.g. a group of checkboxes), you'll need to interact with the r.PostForm map directly:

```
for i, item := range r.PostForm["items"] {
	fmt.Fprintf(w, "%d: Item %s\n", i, item)
}
```

POST, PUT, and PATCH request bodies are limited to 10MB if you aren't using enctype="multipart/form-data" (you're sending multipart data), and

r.ParseForm will return an error if the form exceeds this. You can change the limit with http.MaxBytesReader().

Automatic form parsing

You can use packages like go-playground/form or gorilla/schema to automatically decode form data in your structs.

Here's using go-playground/form:

```
// cmd/web/handlers.go

type snippetCreateForm struct {
	Title		string `form:"title"`
	Content		string `form:"content"`
	Expires		int	`form:"expires"`
	validator.Validator `form:"-"`
}


/// ...
func (app *application) snippetCreatePost(w http.ResponseWriter, r *http.Request) {
	// ...
	var form snippetCreateForm

	// app is our application struct and has a formDecoder.
	if err = app.formDecoder.Decode(&form, r.PostForm); err != nil {
		app.clientError(w, http.StatusBadRequest)
		return
	}
	// ...
```

You'll need some additional error handling, though. If the decoder doesn't get a non-nil pointer as the target decode destination, it'll return a form.InvalidDecoderError. This is a critical error, so we should handle it!

```go
// cmd/web/helpers.go
func (app *application) decodePostForm(r *http.Request, dst any) error {
	err := r.ParseForm()
	if err != nil {
		return err
	}

	if err := app.formDecoder.Decode(dst, r.PostForm); err != nil {
		var invalidDecoderError *form.InvalidDecoderError
		if errors.As(err, &invalidDecoderError) {
			panic(err)
		}

		return err
	}

	return nil
}

// cmd/web/handlers.go
func (app *application) snippetCreatePost(w http.ResponseWriter, r
*http.Request) {
	var form snippetCreateForm

	if err := app.decodePostForm(r, &form); err != nil {
		app.clientError(w, http.StatusBadRequest)
		return
	}
```

```
	form.CheckField(validator.NotBlank(form.Title), "title", "Title
cannot be blank")
	form.CheckField(validator.MaxChars(form.Title, 100), "title", "Title
cannot be longer than 100 characters")
	form.CheckField(validator.NotBlank(form.Content), "content",
"Content cannot be blank")
	form.CheckField(validator.PermittedValue(form.Expires, 1, 7, 365),
"expires", "Expiry must be 1, 7, or 365 days")

	if !form.Valid() {
		data := app.newTemplateData(r)
		data.Form = form
		app.render(w, r, http.StatusUnprocessableEntity,
"create.tmpl", data)
		return
	}

	id, err := app.snippets.Insert(form.Title, form.Content, form.Expires)
	if err != nil {
		app.serverError(w, r, err)
		return
	}

	http.Redirect(w, r, fmt.Sprintf("/snippet/view/%d", id),
http.StatusSeeOther)
}
```

Sessions

There are a lot of security considerations when it comes to sessions, and they aren't easy to implement. Instead of rolling your own, prefer an existing, well-tested solution.

- **gorilla/sessions is good**
- **alexedwards/scs (made by author) is also good, plus it lets you renew session IDs**

We're using alexedwards/scs here.

Start by creating a sessions table in your MySQL database.

```
USE snippetbox;

CREATE TABLE sessions (
  token CHAR(43) PRIMARY KEY,
  data BLOB NOT NULL,
  expiry TIMESTAMP(6) NOT NULL
);

CREATE INDEX sessions_expiry_idx ON sessions (expiry);
```

- **token is a unique, randomly generated ID for each session**
- **data contains session data you'd like to share between HTTP requests, stored as binary data in a binary large object (BLOB) type**
- **expiry is the expiry time, which scs uses to automatically delete expired sessions to avoid the table growing too large**

To create it and use it throughout your app, we'll add it to the application struct we've been using.

```
// cmd/web/main.go
// ...
type application struct {
	logger        *slog.Logger
	snippets      *models.SnippetModel
```

```
	templateCache  map[string]*template.Template
	formDecoder    *form.Decoder
	sessionManager *scs.SessionManager // <-- here
}

func main() {
	// ...
	sessionManager := scs.New()  // initialize
	sessionManager.Store = mysqlstore.New(db)  // use MySQL db
	sessionManager.Lifetime = 12 * time.Hour  // lifetime of 12h
	// ... | add it to your `application` struct below
}
```

To store sessions with the session manager, you'll want to employ its middleware on the routes you'd like to store sessions for.

```
// cmd/web/routes.go as part of routes()
dynamic := alice.New(app.sessionManager.LoadAndSave)

router.HandlerFunc(http.MethodGet, "/",
dynamic.ThenFunc(app.home).ServeHTTP)
router.HandlerFunc(http.MethodGet, "/snippet/view/:id",
dynamic.ThenFunc(app.snippetView).ServeHTTP)
router.HandlerFunc(http.MethodGet, "/snippet/create",
dynamic.ThenFunc(app.snippetCreate).ServeHTTP)
router.HandlerFunc(http.MethodPost, "/snippet/create",
dynamic.ThenFunc(app.snippetCreatePost).ServeHTTP)
```

And you can store things in the sessions with e.g.:

```
// Put this in your handlers, etc.
// Write 'Successfully created' string with 'flash' key in session data
```

app.sessionManager.Put(r.Context(), "flash", "Snippet successfully created!")

Behind the scenes of session management

The book describes how, if you check your network tab in your browser console, you'll see a cookie named session. This cookie includes your session token, which is a high-entropy random string.

In your session store (here: a MySQL database), there will be a corresponding data entry which stores the key and session data, as well as the expiry time for the session.
The MySQL BLOB contains a gob-encoded representation of the session data.

Whenever you make changes to your session data, the data is updated in the database accordingly.

Whenever there's an incoming request, the LoadAndSave middleware checks for a session cookie. If it's present, it'll read the token and retrieve the corresponding session data from the database, while also checking that the session hasn't yet expired. It'll then add the session data to the request context we've used in our handler above.

If you make changes to the session data, LoadAndSave updates the database before it returns. If you recall, LoadAndSave, being middleware, is just part of the handler chain. It's encountered first on the way down the chain, where it adds the session data, and then again up the chain, where it updates the database with any modifications made down the chain.

The book did not describe how the cookie would initially be established with the browser. So I did some research to satiate my curiosity:

1. **User visits your website for the first time. They make their first request, so there's no session cookie present.**
2. **When this request is received on the server, it checks that there's isn't an existing session associated with the user. Then it generates a new session by initializing an unique session ID and the session data.**
3. **Store the session data in the session store (in our case, a MySQL database) with the session ID serving as a key.**
4. **Respond to the first-time request by setting a Set-Cookie header in the response, which will include a session cookie (session) and the unique session ID as a value. There's often additional directives for**

the cookie, e.g. HttpOnly, Secure, expiry time, and the path and domain for which the cookie is valid.

5. In subsequent requests from the browser to the server, the browser will include the session cookie with the request headers.

And that leads us back to the description above, regarding the flow after the cookie has been established.

Server and security

It’s common to customize your server by creating a http.Server struct, instead of using http.ListenAndServe:

```
// Basic example

srv := &http.Server{

	Addr:	*addr,

	Handler:	app.routes(),

}


if err := srv.ListenAndServe(); err != nil {

	// ...

}
```

For example, the http.Server can write its log entries regarding things like unrecovered panics, problems accepting or writing to HTTP connections, and so on.
These entries use the standard logger, which isn’t desirable if you use your own logger.

```
// I'm already using this:

logger := slog.New(slog.NewTextHandler(os.Stdout, &slog.HandlerOptions{}))


// So we can have http.Server use it as well:

srv := &http.Server{

	// ...
```

```
        ErrorLog: slog.NewLogLogger(logger.Handler(), slog.LevelError),
}
```

Generating a self-signed TLS certificate

This is how we switch from HTTP to HTTPS.

HTTPS is HTTP sent over a Transport Layer Security (TLS) connection. The benefit is that HTTPS traffic is encrypted and signed, helping ensure privacy and integrity during transit.

TLS is the modern version of Secure Sockets Layer (SSL).

For your server to use TLS, you'll need to generate a TLS certificate. Production servers can use Let's Encrypt, but for development purposes, you can generate your own self-signed certificate.
That's the same as a normal TLS certificate, except it isn't cryptographically signed by a trusted certificate authority.

If you've used homebrew like me:

```
go run
/home/linuxbrew/.linuxbrew/Cellar/go/1.22.0/libexec/src/crypto/tls/generate
_cert.go --rsa-bits=2048 --host=localhost
```

The path is likely /usr/local/go/src/crypto/tls/generate_cert.go otherwise.

This generates a 2048-bit RSA key pair and stores it in a key.pem file, as well as generates a self-signed TLS certificate for the localhost host with the public key, stored in cert.pem. Both of these files are PEM encoded, which is the standard format used by most TLS implementations.

To run your application as a HTTPS server, all you need to do is:

```
// ...


// Ensure your session manager sets the Secure attribute, meaning the
cookie is only sent to a user's browser when a HTTPS connection is used

sessionManager.Cookie.Secure = true


// ...
```

```
// Start the server & pass in the TLS certificate and private key paths:
if err := srv.ListenAndServeTLS("./tls/cert.pem", "./tls/key.pem"); err != nil {
	// ...
}
```

HTTPS settings

Go has good default settings for its HTTPS server, but you can customize them.
It is often a good idea to restrict the elliptic curves that can be used during the TLS handshake.

Update your http.Server struct like so:

```
tlsConfig := &tls.Config{
	CurvePreferences: []tls.CurveID{tls.X25519, tls.CurveP256},
}


srv := &http.Server{
	Addr:     *addr,
	Handler:  app.routes(),
	ErrorLog: slog.NewLogLogger(logger.Handler(), slog.LevelError),
	TLSConfig: tlsConfig,
}
```

You can also restrict which cipher suites are used, as well as the min and max versions of TLS.

Connection timeouts

You can improve the resiliency of your server by adding timeout settings:

```
srv := &http.Server{
	// ...
	IdleTimeout: time.Minute,
	ReadTimeout: 5 * time.Second,
```

WriteTimeout: 10 * time.Second,

}

These are server-wide settings that act on the underlying connection and apply to all requests.

- **IdleTimeout: automatically closes all keep-alive connections after 1 minute of inactivity.**
    - **Go enables keep-alive on all accepted connections by default. This helps reduce latency (especially for HTTPS connections), because a client can reuse the same connection multiple times, without repeating the TLS handshake. By default, these keep-alives close after a few minutes, to clear up connections where the user disappeared unexpectedly.**
- **ReadTimeout: If the request headers or body are still being read 5 seconds after the request is first accepted, Go closes the underlying connection. This is a hard closure, meaning the user doesn't get any HTTP(S) response.**
    - **This helps mitigate risk from slow-client attacks (e.g. Slowloris) which could otherwise keep a connection open forever by sending partial, incomplete HTTP(S) requests.**
- **WriteTimeout: Closes the underlying connection if the server attempts to write to the connection after a given period (here, 10s). Behaves differently depending on the protocol.**
    - **HTTP: if data is written to the connection more than 10s after the read of the request header is finished, Go closes the connection instead of writing.**
    - **HTTPS: if some data is written to the connection more than 10s after the request is first accepted, Go closes the underlying connection instead of writing the data. So if you use HTTPS, it's a good idea to set WriteTimeout to a value greater than ReadTimeout.**
    - **Writes by a handler are buffered and written to the connection as one when the handler returns. So this isn't to prevent long-running handlers but to prevent the data that the handler returns from being too long to write.**

Adding authentication

First, add handlers and corresponding routes. Something like this:

```
// cmd/web/routes.go

// ...

router.Handler(http.MethodGet, "/user/signup",
dynamic.ThenFunc(app.userSignup))

router.Handler(http.MethodPost, "/user/signup",
dynamic.ThenFunc(app.userSignupPost))

router.Handler(http.MethodGet, "/user/login",
dynamic.ThenFunc(app.userLogin))

router.Handler(http.MethodPost, "/user/login",
dynamic.ThenFunc(app.userLoginPost))

router.Handler(http.MethodPost, "/user/logout",
dynamic.ThenFunc(app.userLogoutPost))

// ...
```

You'll want to store your user information in your database.

```
USE snippetbox;


CREATE TABLE users (
  id INTEGER NOT NULL PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL,
  hashed_password CHAR(60) NOT NULL,
  created DATETIME NOT NULL
);


ALTER TABLE users ADD CONSTRAINT users_uc_email UNIQUE
(email);
```

In practice, you probably wouldn't want an auto-incrementing ID for your user table.

hashed_password is CHAR(60) because we're storing bcrypt hashes that are exactly 60 characters long.

Your database can never store plain-text versions of your users' passwords, which is why we store a one-way hash of the password. This hash is derived with a computationally expensive key-derivation function like bcrypt (or Argon2, scrypt).
For bcrypt, Go has helper functions specifically for hashing and checking passwords. Grab it with go get golang.org/x/crypto/bcrypt@latest.

Here's how you can generate a hash of a plain-text password:

```
hash, err := bcrypt.GenerateFromPassword([]byte("password"), 12)
```

The resulting hash would be something like

$2a$12$1iMBMO.3TgGu9l7UgL69T.cTPyLIPXnFxy.MAuIIMujdNzXPUvDE

The second parameter (12 used above) is an integer between 4 and 31 and represents the cost. Using 12 means bcrypt uses

iterations to generate the password hash. The higher the cost, the more expensive the hash is for an attacker to crack, but also for your app to create the hash when a user signs up.
12 is a reasonable minimum, but you may want to set it higher.

You can then check if a plain-text password matches a particular hash:

```
hash :=
[]byte("$2a$12$1iMBMO.3TgGu9l7UgL69T.cTPyLIPXnFxy.MAuIIMujdNzXPUvDE")

err := bcrypt.CompareHashAndPassword(hash, []byte("password"))
```

which returns nil if they match, and an error if they don't.

Some databases provide built-in functions for password hashing and verification, but you should avoid those. They tend to be vulnerable to side-channel timing attacks because string comparison time isn't constant, and sending plain-text passwords to your database risks the password being stored in your database logs.

The user data access layer in your Go code will look like this:

```
// internal/models/users.go

package models
```

```
import (
	"database/sql"
	"errors"
	"strings"
	"time"

	"github.com/go-sql-driver/mysql"
	"golang.org/x/crypto/bcrypt"
)

type User struct {
	ID             int
	Name           string
	Email          string
	HashedPassword []byte
	Created        time.Time
}

type UserModel struct {
	DB *sql.DB
}

func (m *UserModel) Insert(name, email, password string) error {
	hashedPassword, err :=
bcrypt.GenerateFromPassword([]byte(password), 12)
	if err != nil {
		return err
```

```
	}

	stmt := `INSERT INTO users (name, email, hashed_password, created)
	VALUES(?, ?, ?, UTC_TIMESTAMP())`

	if _, err = m.DB.Exec(stmt, name, email, string(hashedPassword)); err != nil {
		var mySQLError *mysql.MySQLError
		if errors.As(err, &mySQLError) {
			if mySQLError.Number == 1062 && strings.Contains(mySQLError.Message, "users_uc_email") {
				return ErrDuplicateEmail // custom error
			}
		}
	}

	return nil
}

func (m *UserModel) Authenticate(email, password string) (int, error) {
	var (
		id int
		hashedPassword []byte
	)

	stmt := `SELECT id, hashed_password FROM users WHERE email = ?`
```

```
	if err := m.DB.QueryRow(stmt, email).Scan(&id,
&hashedPassword); err != nil {
		if errors.Is(err, sql.ErrNoRows) {
			return 0, ErrInvalidCredentials
		}

		return 0, err
	}

	if err := bcrypt.CompareHashAndPassword(hashedPassword,
[]byte(password)); err != nil {
		if errors.Is(err, bcrypt.ErrMismatchedHashAndPassword) {
			return 0, ErrInvalidCredentials
		}

		return 0, err
	}

	return id, nil
}

func (m *UserModel) Exists(id int) (bool, error) {
	var exists bool

	stmt := `SELECT EXISTS(SELECT 1 FROM users WHERE id = ?)`

	err := m.DB.QueryRow(stmt, id).Scan(&exists)
	return exists, err
```

```
}
```

Your user sign-up handler should include something like this:

```
// cmd/web/handlers.go
// ...
func (app *application) userSignupPost(w http.ResponseWriter, r *http.Request) {
	// ... Form decoding, validation above ...
	if err := app.users.Insert(form.Name, form.Email, form.Password); err != nil {
		if errors.Is(err, models.ErrDuplicateEmail) {
			form.AddFieldError("email", "Address is already in use")
			data := app.newTemplateData(r)
			data.Form = form
			app.render(w, r, http.StatusUnprocessableEntity, "signup.tmpl", data)
		} else {
			app.serverError(w, r, err)
		}

		return
	}

	app.sessionManager.Put(r.Context(), "flash", "Your signup was successful. Please log in.")

	http.Redirect(w, r, "/user/login", http.StatusSeeOther)
}
```

Your user login handler should include something like this:

```
// cmd/web/handlers.go
// ...
func (app *application) userLoginPost(w http.ResponseWriter, r
*http.Request) {
	// ... Form decoding, validation above ...
	id, err := app.users.Authenticate(form.Email, form.Password)
	if err != nil {
		if errors.Is(err, models.ErrInvalidCredentials) {
			form.AddNonFieldError("Email or password is
incorrect")

			data := app.newTemplateData(r)
			data.Form = form
			app.render(w, r, http.StatusUnprocessableEntity,
"login.tmpl", data)
		} else {
			app.serverError(w, r, err)
		}

		return
	}

	// Change the session token to prevent session fixation attacks
	// Good to do when auth state or privileges  levels change (e.g. login,
logout, change password, etc.)
	if err = app.sessionManager.RenewToken(r.Context()); err != nil {
		app.serverError(w, r, err)
		return
	}
```

```
	app.sessionManager.Put(r.Context(), "authenticatedUserID", id)
	http.Redirect(w, r, "/", http.StatusSeeOther)
}
```

Even though we renew the token, which changes the ID of the current user's session, it retains the data associated with the session.

Logout is very simple to handle:

```
// cmd/web/handlers.go
// ...
func (app *application) userLogoutPost(w http.ResponseWriter, r *http.Request) {
	if err := app.sessionManager.RenewToken(r.Context()); err != nil {
		app.serverError(w, r, err)
		return
	}

	app.sessionManager.Remove(r.Context(), "authenticatedUserID")
	app.sessionManager.Put(r.Context(), "flash", "You've been logged out successfully!")
	http.Redirect(w, r, "/", http.StatusSeeOther)
}
```

Authorization

We want to use the authentication for allowing and disallowing certain things across our application. This is authorization.
For example, we want only logged-in users to be able to create new snippets. We'd also like the contents of the navigation bar to change, depending on whether you are logged in or not.

```
// cmd/web/helpers.go
// ...
func (app *application) isAuthenticated(r *http.Request) bool {
```

```
	// This isn't very robust. See "Request context" section for more.
	return app.sessionManager.Exists(r.Context(),
"authenticatedUserID")
}
```

If you want to use this state across your application, you'll want somewhere to share it from. Throughout the building of this system, we've been using a templateData struct to hold data for our .tmpl templates, which replace the template placeholders and then send the rendered template as HTML to the browser.

```
type templateData struct {
	CurrentYear     int
	Snippet         models.Snippet
	Snippets        []models.Snippet
	Form            any
	Flash           string
	IsAuthenticated bool
}
```

I haven't been noting anything about our usage of templates because I have little interest in using Go to server-render HTML. I'd rather use tools like Next.js, Remix, Vite. This would allow for Server-Side Rendering (SSR) still, and it's much easier to build user interfaces with e.g. React or Svelte. Using Go templates is not bad, it's just not something I'm interested in as of reading this book.

You can restrict access with middleware like this:

```
func (app *application) requireAuthentication(next http.Handler)
http.Handler {
	return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
		// Redirect to the login page if the user is not authenticated
		// Early return to prevent subsequent handlers from executing
		if !app.isAuthenticated(r) {
```

```
                http.Redirect(w, r, "/user/login", http.StatusSeeOther)
                return
            }

            // Ensure pages that require authentication aren't cached
            w.Header().Add("Cache-Control", "no-store")

            // Call the next handler in the chain
            next.ServeHTTP(w, r)
        })
}
```

Adding this middleware to our router is simple:

```
package main

import (
        "net/http"

        "github.com/julienschmidt/httprouter"
        "github.com/justinas/alice"
)

func (app *application) routes() http.Handler {
        router := httprouter.New()

        router.NotFound = http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
            app.notFound(w)
        })
```

```
	fileServer := http.FileServer(http.Dir("./ui/static/"))
	router.Handler(http.MethodGet, "/static/*filepath",
http.StripPrefix("/static", fileServer))

	dynamic := alice.New(app.sessionManager.LoadAndSave)

	router.HandlerFunc(http.MethodGet, "/",
dynamic.ThenFunc(app.home).ServeHTTP)
	router.HandlerFunc(http.MethodGet, "/snippet/view/:id",
dynamic.ThenFunc(app.snippetView).ServeHTTP)

	router.Handler(http.MethodGet, "/user/signup",
dynamic.ThenFunc(app.userSignup))
	router.Handler(http.MethodPost, "/user/signup",
dynamic.ThenFunc(app.userSignupPost))
	router.Handler(http.MethodGet, "/user/login",
dynamic.ThenFunc(app.userLogin))
	router.Handler(http.MethodPost, "/user/login",
dynamic.ThenFunc(app.userLoginPost))

	protected := dynamic.Append(app.requireAuthentication)
	router.HandlerFunc(http.MethodGet, "/snippet/create",
protected.ThenFunc(app.snippetCreate).ServeHTTP)
	router.HandlerFunc(http.MethodPost, "/snippet/create",
protected.ThenFunc(app.snippetCreatePost).ServeHTTP)
	router.Handler(http.MethodPost, "/user/logout",
protected.ThenFunc(app.userLogoutPost))

	standard := alice.New(app.recoverPanic, app.logRequest,
secureHeaders)
```

```
    return standard.Then(router)
}
```

CSRF protection

CRRF stands for cross-site request forgery. This is a type of attack where malicious third-party websites send state-changing HTTP requests to your website.

CSRF attack process

1. **User logs in. Session cookie persists for 12 hours, so they stay logged in even after navigating away.**
2. **User goes to a malicious site, which sends a cross-site request to our POST /snippet/create endpoint, to add a new snippet to our database. The user's session cookie for our application is sent along with the request.**
3. **We interpret the request as coming from the logged-in user because it has the session cookie, so we process the request with that user's privileges.**

So we added a new snippet to our database, thinking the user intended to do so. However, this is completely unknown to them.

One way to mitigate CSRF attacks is ensuring SameSite is appropriately set on the session cookie.
alexedwards/scs sets SameSite=Lax, so the session cookie won't be sent by the user's browser for any cross-site POST, PUT, or DELETE requests. That means we'll be fine as long as we use POST for any state-changing HTTP requests, as the session cookie isn't sent on those requests if they come from another site.
It isn't necessarily better to set SameSite=Strict, because then their session cookie would not be sent when clicking on your site from another (via GET or HEAD), so they would be treated as not logged in, despite having an active session.
However, SameSite is new and not fully supported.

For that reason, we need some form of token check. The two most popular packages for preventing CSRF attacks in Go web apps are gorilla/csrf and justinas/nosurf.
Both use the double-submit cookie pattern to prevent attacks. Here, a

random CSRF token is generated and sent to the user in a CSRF cookie. This token is then added to a hidden field in each HTML form that is potentially vulnerable to CSRF. When the form is submitted, both packages use middleware to check the hidden field value and cookie value match.

We use justinas/nosurf. It's self-contained and doesn't have any additional dependencies.

Create middleware noSurf:

```
// cmd/web/middleware.go

func noSurf(next http.Handler) http.Handler {

	csrfHandler := nosurf.New(next)

	csrfHandler.SetBaseCookie(http.Cookie{

		HttpOnly: true,

		Path:     "/",

		Secure:   true,

	})


	return csrfHandler

}
```

Update your routes() to use noSurf:

```
// cmd/web/routes.go

func (app *application) routes() http.Handler {

	// ...

	dynamic := alice.New(app.sessionManager.LoadAndSave, noSurf)

	// ...

}
```

To make form submissions work, we need the CSRF token from nosurf.Token() and add it to a hidden csrf_token field in each of our forms.

First, add CSRFToken to templateData:

```
type templateData struct {
```

```
	CurrentYear	int
	Snippet		models.Snippet
	Snippets	[]models.Snippet
	Form		any
	Flash		string
	IsAuthenticated bool
	CSRFToken	string
}
```

We have a constructor function for this struct that we've been using:

```
// cmd/web/helpers.go
// ...
func (app *application) newTemplateData(r *http.Request) templateData {
	return templateData{
		CurrentYear:	time.Now().Year(),
		Flash:		app.sessionManager.PopString(r.Context(),
"flash"),
		IsAuthenticated: app.isAuthenticated(r),
		CSRFToken:	nosurf.Token(r),
	}
}
// ...
```

Now you'll want to embed this in every form. {{.CSRFToken}} is the template placeholder for the token. Put this in every vulnerable form:

```
<input type="hidden" name="csrf_token" value="{{.CSRFToken}}" />
```

Request context

We can make the isAuthenticated check more robust by querying the users database to ensure authenticatedUserID is real and valid.
But that would be slow because isAuthenticated can be called multiple times in each request cycle.

Instead, it would be better to do the check in some middleware to determine if the current request is from an authenticated user or not, and then pass the information down to all subsequent handlers in the chain. That's doable with request context.

Every http.Request that our middleware and handlers process has a context.Context object, which we can use to store information during the lifetime of the request.
In this case, we'd want to check if a user is authenticated once, and let all subsequent middleware and handlers know.

Generally, you can add information to a request's context like so:

```
ctx := r.Context() // get existing context from the *http.Request `r`

ctx = context.WithValue(ctx, "isAuthenticated", true) // create new copy that contains "isAuthenticated" with a value of `true`

r = r.WithContext(ctx) // create a copy of the request and assign to request


// In practice, you'd do:

ctx = context.WithValue(r.Context(), "isAuthenticated", true)

r = r.WithContext(ctx)
```

And you can retrieve the values again. The values are stored with type any, so assert them to their original types before use:

```
isAuthenticated, ok := r.Context().Value("isAuthenticated").(bool)

if !ok {

	return errors.New("could not convert value to bool")

}
```

You need to avoid key collisions. To do so, you could create your own custom type to use for your context keys.

```
type contextKey string

const isAuthenticatedContextKey = contextKey("isAuthenticated")


ctx = context.WithValue(r.Context(), isAuthenticatedContextKey, true)

r = r.WithContext(ctx)
```

// And similar for getting the value.

Request context for authentication

So let’s add the context key to our codebase:

```
// cmd/web/context.go
package main

type contextKey string

const isAuthenticatedContextKey = contextKey("isAuthenticated")
```

And implement the authenticate middleware:

```
func (app *application) authenticate(next http.Handler) http.Handler {
	return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
		id := app.sessionManager.GetInt(r.Context(), "authenticatedUserID")
		if id == 0 {
			next.ServeHTTP(w, r)
			return
		}

		exists, err := app.users.Exists(id)
		if err != nil {
			app.serverError(w, r, err)
			return
		}

		if exists {
```

```
                ctx := context.WithValue(r.Context(),
isAuthenticatedContextKey, true)
                r = r.WithContext(ctx)
            }

            next.ServeHTTP(w, r)
        })
}
```

The user IDs here are auto-incrementing integers in the database, which is why we use integers. The default zero value for ints in Go is 0, so that's what is returned if there is no authenticatedUserID in this session. If that's the case, the user just isn't logged in, and we'll call the next handler to go down the handler chain, and return when we get back up.
We don't want to change the *http.Request when we don't have a valid authenticated user.
But when we do, we create a new, modified request and pass it down the chain with isAuthenticatedContextKey set to true.

Now we can add it to our middleware:

```
// cmd/web/routes.go

func (app *application) routes() http.Handler {
    // ...
    dynamic := alice.New(app.sessionManager.LoadAndSave, noSurf,
app.authenticate)
    // ...
}
```

And finally, we update the isAuthenticated() helper:

```
// cmd/web/helpers.go
// ...
func (app *application) isAuthenticated(r *http.Request) bool {
```

```
	isAuthenticated, ok :=
r.Context().Value(isAuthenticatedContextKey).(bool)
	if !ok {
		return false
	}

	return isAuthenticated
}
```

Make sure request context is only ever used to store information relevant to the lifetime of a specific request. Don't use it to pass e.g. loggers or database connection pools.
Either use an application struct like we have in this book or use closures.

File embedding

You can use the embed standard library package to embed external files into your Go program, meaning your program is self-contained and has all it needs to run as part of its compiled binary executable.

```
// ui/efs.go
package ui

import (
	"embed"
)

//go:embed "html" "static"
var Files embed.FS
```

//go:embed "static" is a special comment directive that instructs Go to store the files in ui/static in an embedded filesystem referenced by Files when the app is compiled.

Now update your routes():

```
// cmd/web/routes.go
```

```
package main

import (
	"net/http"

	"github.com/chhoumann/snippetbox/ui"
	"github.com/julienschmidt/httprouter"
	"github.com/justinas/alice"
)

func (app *application) routes() http.Handler {
	// ...
	fileServer := http.FileServer(http.FS(ui.Files))
	router.Handler(http.MethodGet, "/static/*filepath", fileServer)
	// ...
}
```

This also makes it easier to serve our templates:

```
// cmd/web/templates.go
package main

import (
	"html/template"
	"io/fs"
	"path/filepath"
	"time"

	"github.com/chhoumann/snippetbox/internal/models"
```

```
	"github.com/chhoumann/snippetbox/ui"
)

type templateData struct {
	CurrentYear     int
	Snippet         models.Snippet
	Snippets        []models.Snippet
	Form            any
	Flash           string
	IsAuthenticated bool
	CSRFToken       string
}

func humanDate(t time.Time) string {
	return t.Format("02 Jan 2006 at 15:04")
}

var functions = template.FuncMap{
	"humanDate": humanDate,
}

func newTemplateCache() (map[string]*template.Template, error) {
	cache := map[string]*template.Template{}

	pages, err := fs.Glob(ui.Files, "htp/pages/*.tmpl")
	if err != nil {
		return nil, err
```

```
	}

	for _, page := range pages {
		name := filepath.Base(page)

		patterns := []string{
			"html/base.tmpl",
			"html/partials/*.tmpl",
			page,
		}

		ts, err :=
template.New(name).Funcs(functions).ParseFS(ui.Files, patterns...)
		if err != nil {
			return nil, err
		}

		cache[name] = ts
	}

	return cache, nil
}
```

So when you want to deploy your web application, you'd

- go build -o /tmp/web ./cmd/web/ to build your app to /tmp/web
- cp -r ./tls /tmp/ to copy your certificate to tmp
- cd /tmp/
- ./web to run

Testing

Table-driven testing is a nice practice. You define the test cases you'd like to go through as a 'table', and then iterate over them. In Go, you'd use anonymous structs:

```
// cmd/web/templates_test.go

package main


import (
	"testing"
	"time"
)


func TestHumanDate(t *testing.T) {


	tests := []struct {
		name string
		tm   time.Time
		want string
	}{
		{
			name: "UTC",
			tm:   time.Date(2024, time.February, 17, 13, 46, 0, 0,
time.UTC),
			want: "17 Feb 2024 at 13:46",
		},
		{
			name: "Empty",
			tm:   time.Time{},
			want: "",
```

```
		},
		{
			name: "CET",
			tm:   time.Date(2024, time.February, 17, 13, 46, 0, 0, time.FixedZone("CET", 60*60)),
			want: "17 Feb 2024 at 12:46", // CET is 1 hour ahead of UTC
		},
	}

	for _, tt := range tests {
		t.Run(tt.name, func(t *testing.T) {
			got := humanDate(tt.tm)

			assert.Equal(t, hd, tt.want) // see below
		})
	}
}
```

Here’s a nice helper to check for equality during testing:

```
// internal/assert/assert.go
package assert

import (
	"testing"
)

func Equal[T comparable](t *testing.T, got, want T) {
	// This line is needed to tell the test suite that this method is a helper.
```

```
	// This way when it fails the line number reported will be in our function call
	// rather than inside this helper (which is not helpful).
	t.Helper()

	if got != want {
		t.Errorf("got %v | want %v", got, want)
	}
}
```

Testing HTTP handlers

```
// cmd/web/handlers_test.go
package main

import (
	"bytes"
	"io"
	"net/http"
	"net/http/httptest"
	"testing"

	"github.com/chhoumann/snippetbox/internal/assert"
)

func TestPing(t *testing.T) {
	rr := httptest.NewRecorder()

	r, err := http.NewRequest(http.MethodGet, "/ping", nil)
	if err != nil {
```

```
		t.Fatal(err) // doesn't make sense to continue if we can't create a request
	}

	ping(rr, r) // w.Write([]byte("OK"))

	rs := rr.Result()
	assert.Equal(t, rs.StatusCode, http.StatusOK)

	defer rs.Body.Close()
	body, err := io.ReadAll(rs.Body)
	if err != nil {
		t.Fatal(err) // doesn't make sense to continue if we can't read the body
	}

	body = bytes.TrimSpace(body)
	assert.Equal(t, string(body), "OK")
}
```

Testing middleware

```
// cmd/web/middleware_test.go
package main

import (
	"bytes"
	"io"
	"net/http"
	"net/http/httptest"
```

```
	"testing"

	"github.com/chhoumann/snippetbox/internal/assert"
)

func TestSecureHeaders(t *testing.T) {
	rr := httptest.NewRecorder()

	r, err := http.NewRequest(http.MethodGet, "/", nil)
	if err != nil {
		t.Fatal(err)
	}

	next := http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
		w.Write([]byte("OK"))
	})

	secureHeaders(next).ServeHTTP(rr, r)

	rs := rr.Result()

	expectedValue := "default-src 'self'; style-src 'self'
fonts.googleapis.com; font-src fonts.gstatic.com"
	assert.Equal(t, rs.Header.Get("Content-Security-Policy"),
expectedValue)

	expectedValue = "origin-when-cross-origin"
```

```
	assert.Equal(t, rs.Header.Get("Referrer-Policy"), expectedValue)

	expectedValue = "nosniff"
	assert.Equal(t, rs.Header.Get("X-Content-Type-Options"),
expectedValue)

	expectedValue = "deny"
	assert.Equal(t, rs.Header.Get("X-Frame-Options"), expectedValue)

	expectedValue = "0"
	assert.Equal(t, rs.Header.Get("X-XSS-Protection"), expectedValue)

	assert.Equal(t, rs.StatusCode, http.StatusOK)

	defer rs.Body.Close()
	body, err := io.ReadAll(rs.Body)
	if err != nil {
		t.Fatal(err)
	}

	body = bytes.TrimSpace(body)
	assert.Equal(t, string(body), "OK")
}
```

End-to-end testing

```
// cmd/web/handlers_test.go
package main

import (
```

```
	"bytes"
	"io"
	"log/slog"
	"net/http"
	"net/http/httptest"
	"testing"

	"github.com/chhoumann/snippetbox/internal/assert"
)

func TestPing(t *testing.T) {
	app := &application{
		logger: slog.New(slog.NewTextHandler(io.Discard, nil)),
	}

	ts := httptest.NewTLSServer(app.routes()) // if testing a http server,
use httptest.NewServer instead
	defer ts.Close()

	rs, err := ts.Client().Get(ts.URL + "/ping")
	if err != nil {
		t.Fatal(err)
	}

	assert.Equal(t, rs.StatusCode, http.StatusOK)

	defer rs.Body.Close()
	body, err := io.ReadAll(rs.Body)
```

```
	if err != nil {
		t.Fatal(err) // doesn't make sense to continue if we can't read the body
	}

	body = bytes.TrimSpace(body)
	assert.Equal(t, string(body), "OK")
}
```

But you can simplify this quite a bit by creating some test utility functions:

```
// cmd/web/testutils_test.go
package main

import (
	"io"
	"log/slog"
	"net/http"
	"net/http/httptest"
	"testing"
)

func newTestApplication(t *testing.T) *application {
	return &application{
		logger: slog.New(slog.NewTextHandler(io.Discard, nil)),
	}
}

type testServer struct {
	*httptest.Server
```

```
}

func newTestServer(t *testing.T, h http.Handler) *testServer {
	ts := httptest.NewServer(h)

	jar, err := cookiejar.New(nil)
	if err != nil {
		t.Fatal(err)
	}

	// Set the client to use the cookie jar; this will store cookies sent by the server.
	ts.Client().Jar = jar

	// Disable redirect-following for the test client. This means that if the server sends
	// a 3xx response status code, the client will return the response instead of following
	// the redirect.
	ts.Client().CheckRedirect = func(req *http.Request, via []*http.Request) error {
		return http.ErrUseLastResponse
	}

	return &testServer{ts}
}

func (ts *testServer) get(t *testing.T, urlPath string) (int, http.Header, string) {
```

```
	rs, err := ts.Client().Get(ts.URL + urlPath)
	if err != nil {
		t.Fatal(err)
	}

	defer rs.Body.Close()
	body, err := io.ReadAll(rs.Body)
	if err != nil {
		t.Fatal(err)
	}

	return rs.StatusCode, rs.Header, string(body)
}
```

Liked these notes? Join the newsletter.

Get notified whenever I post new notes.